

SPINE: Stochastic Plastic Intelligence through Network Evolution

A research proposal for AI systems that learn not only their parameters, but their computation

Working whitepaper - October 2026

Abstract

Current foundation models primarily learn by changing numerical parameters inside a largely fixed computational graph. Modern systems have added conditional routing, mixture-of-experts, test-time training, search, tool use, and verifier loops, but these mechanisms are usually layered around a fixed substrate. This paper proposes **SPINE**, a Stochastic Plastic Intelligence Network: a neural system in which weights, connectivity probabilities, computational pathways, and long-lived structural specializations are learned jointly.

SPINE treats the network itself as a stochastic object. A forward pass samples a computational graph from a learned distribution over connections, modules, and reasoning pathways. A deliberately exploratory generator produces diverse candidate internal computations; an independently optimized verifier scores their validity, usefulness, novelty, and external evidence; successful structures receive greater probability and unsuccessful structures decay. Architecture changes happen on a slower timescale than ordinary weight updates, while persistent memories and fast test-time adaptation operate on additional timescales.

The proposal unifies established ideas including stochastic gates, Concrete/Gumbel-Softmax relaxations, dynamic sparse training, structural plasticity, stochastic architecture search, mixture-of-experts routing, adaptive computation, test-time training, Monte Carlo search, and generator-verifier systems. The proposed contribution is the synthesis and experimental program: treat **topology distribution, stochastic exploration, verification, and continual learning as one coupled optimization problem** rather than independent add-ons.

The paper makes a stronger claim than ordinary pruning: the objective is not to find a sparse version of a pre-existing network, but to let the network discover and retain useful computational structures. True physical randomness is proposed as an optional source of irreducible entropy, but a central falsifiable prediction is that physical randomness will matter only if the architecture gives randomness an exploitable role; otherwise pseudorandomness should be equivalent.

SPINE is an AGI-oriented research hypothesis, not a claim that this architecture is sufficient for AGI. Its primary scientific question is whether **learning a distribution over computations** provides qualitatively different capability, adaptation, and discovery from learning only the parameters of a fixed computation.

1. Thesis

The dominant deep-learning abstraction is approximately:

Choose a computational graph, then learn its parameters.

SPINE reverses the emphasis:

Learn both the parameters and the distribution over useful computational graphs.

For a conventional network,

$$y = f(x; \theta),$$

where the graph is fixed and θ changes during training.

SPINE introduces a graph variable G :

$$G \sim q_\phi(G \mid x, s),$$

where s is persistent state and ϕ parameterizes a distribution over possible computation graphs. The output becomes

$$y = f(x; \theta, G).$$

Training therefore optimizes both θ and ϕ :

$$\min_{\theta, \phi} \mathbb{E}_{G \sim q_\phi} [L(f(x; \theta, G), y)] + \lambda C(G) - \beta H(q_\phi),$$

with additional terms for diversity, novelty, and verified utility.

This is related to stochastic architecture search, but the intended use is broader: the graph distribution is not merely a temporary search mechanism whose best architecture is exported at the end. **It remains part of the learned system.**

2. What the literature already tells us

The proposal is not built from a vacuum. Several lines of research independently demonstrate that the pieces are viable.

2.1 Stochastic computational units are tractable

The Concrete distribution and Gumbel-Softmax made discrete stochastic decisions trainable with gradient-based methods [1,2]. Louizos et al. used stochastic hard-concrete gates to jointly learn weights and sparsity, allowing network structure to be optimized during training [3]. Variational dropout also demonstrated that learned stochastic gating can produce extremely sparse networks [4].

The important implication is that a neural network need not be a deterministic function of its inputs. A component can represent a **probability of participating in computation**.

2.2 Topology can be changed during learning

Deep Rewiring explicitly rewires sparse networks during supervised training and interprets rewiring as stochastic sampling of network configurations from a posterior [5]. Sparse Evolutionary Training starts from sparse connectivity and changes its topology during learning [6]. RigL demonstrated dynamic sparse training by removing weak connections and regrowing connections using gradient information [7]. Structured RigL extended this toward hardware-friendly structured sparsity and introduced neuron ablation [8].

This is important because the graph itself can be an optimization variable rather than a fixed design decision.

2.3 Architecture distributions can be learned by backpropagation

Stochastic Neural Architecture Search jointly trains architecture-distribution parameters and network parameters using stochastic differentiable decisions [9]. This establishes the mathematical feasibility of optimizing a distribution over structures.

However, classic NAS normally treats architecture discovery as a search process followed by selection of a final architecture. SPINE instead proposes to keep a learned distribution over structural alternatives as a persistent computational capability.

2.4 Conditional computation is already mainstream

Mixture-of-experts models demonstrate that enormous parameter capacity can coexist with much smaller per-token compute by routing tokens to a subset of experts. DeepSeek-V3, for example, reports 671B total parameters and about 37B activated parameters per token [10]. Mixture-of-Depths dynamically allocates computation to selected token positions while respecting an overall compute budget [11].

These methods demonstrate that fixed dense computation is not a law of nature. The system can learn where to spend compute.

SPINE pushes the question one level deeper: **can the system learn what computational resources should exist and how they should connect?**

2.5 Test-time learning turns inference into adaptation

The TTT line of work makes a model continue learning from its current sequence at inference time [12,13]. Titans adds a learned neural long-term memory that updates at test time [14]. In late-2025 work, end-to-end TTT was shown to compress context into model state and maintain constant inference latency with respect to context length [15]. In 2026, TTT-Discover used reinforcement learning at test time for scientific and algorithmic discovery and reported new state-of-the-art solutions using an open model [16].

These results strongly support the proposition that **inference need not be a purely read-only operation**.

2.6 Search plus verification already produces new ideas

FunSearch combines a language model’s generation ability with an evaluator that scores generated programs, and used the loop for mathematical and algorithmic discovery [17]. MCTS-based systems similarly combine candidate generation with search and evaluation [18,19]. The Darwin Godel Machine goes further, iteratively modifying its own agent code, validating changes on benchmarks, and maintaining an archive of alternative agents [20].

This provides strong evidence for a generator-verifier evolutionary loop, although those systems generally evolve external programs, agents, or prompts rather than the internal neural graph itself.

2.7 Recent continual-learning work points to the same bottleneck

A 2026 survey of continual learning in LLMs identifies the static nature of standard pretraining and the difficulty of integrating new knowledge across diverse tasks and timescales as fundamental challenges [21]. A recent prequential test-time learning approach explicitly separates immediate use of new information from persistent trust, requiring later validation before knowledge is promoted to durable guidance [22].

SPINE therefore treats **temporary hypotheses and durable structural changes as different states**, rather than forcing every observation immediately into long-term memory.

3. The missing synthesis

The search did not uncover a demonstrated frontier-scale system that combines all of the following into one general learning loop:

1. stochastic internal computation;

2. learned probabilities over connections or modules;
3. persistent birth/death/rewiring of internal structure;
4. explicit generator-verifier separation;
5. test-time learning from verified experience;
6. an adaptive exploration/entropy controller; and
7. optional irreducible physical entropy as the source of stochastic samples.

This is not proof that no such work exists. It is a bounded literature search, and terminology across fields is inconsistent. The appropriate claim is therefore **research-gap hypothesis**, not novelty proof.

The central proposed synthesis is:

sample computation $\rightarrow$ generate hypotheses $\rightarrow$ verify $\rightarrow$ reinforce useful structures $\rightarrow$ rewire $\rightarrow$ learn again

This turns architecture from a static object into a learned state variable.

4. SPINE architecture

4.1 Four timescales

SPINE separates learning into four timescales.

Fast - activation state. Changes every forward pass. This includes ordinary activations and stochastic microstates.

Medium - weights. Learned with ordinary gradient methods.

Slow - topology probabilities. Connection and module participation probabilities change more slowly, based on accumulated credit and verified outcomes.

Very slow - structural identity. Neurons, modules, expert families, memory slots, and stable subnetworks can be born, die, merge, split, or become specialized.

This separation is crucial. A useful but rare circuit should not disappear because it was inactive for one minibatch.

4.2 Stochastic edges

Each candidate edge $e = (i, j)$ has:

- a numerical weight W_e ;
- a structural logit α_e ;
- a learned existence probability $p_e = \sigma(\alpha_e)$;
- a long-term fitness state F_e .

A forward pass samples

$$z_e \sim \text{Bernoulli}(p_e)$$

and computes

$$y_j = \sum_i z_{ij} W_{ij} x_i.$$

During early training, a Concrete/Gumbel-Softmax relaxation provides gradients. Later, exact hard masks and discrete rewiring events are introduced [1,2,9].

For hardware efficiency, edges should normally be grouped into blocks or N:M patterns rather than arbitrary individual connections; structured sparsity has much better deployment characteristics than unrestricted irregular sparsity [8].

4.3 Stochastic modules

Individual edges are not the only structural random variable. A module can have an activation probability, and a small set of alternative modules can be sampled from a categorical distribution.

Examples:

$$M \sim \text{Categorical}(p_1, \dots, p_k)$$

for selecting a reasoning operator,

or

$$z_m \sim \text{Bernoulli}(p_m)$$

for allowing an entire expert/module to participate.

This creates a hierarchy:

$$\text{edge} < \text{neuron} < \text{module} < \text{strategy}.$$

The system can therefore explore at the smallest useful scale instead of spawning an entirely separate copy of the model for every hypothesis.

5. The generator-verifier loop

5.1 Generator

The generator is intentionally optimized for **useful exploration**, not immediate factuality.

For an input x , it samples K internal trajectories:

$$G_k \sim q_\phi(G \mid x, s, \xi_k),$$

where ξ_k is stochastic entropy.

Each trajectory can differ in:

- activated neurons;
- selected connections;
- expert routing;
- depth;
- memory accesses;
- reasoning strategy;
- local parameter perturbations.

The generator should be allowed to produce ideas that are unlikely but testable.

5.2 Verifier

The verifier scores candidate trajectories rather than generating them.

A generic reward can be written as

$$R_k = R_{task} + \eta R_{novelty} + \gamma R_{evidence} - \lambda C_k - \mu R_{risk}.$$

The strongest verifier is an external one when available:

- code execution;
- theorem proving;
- numerical simulation;
- database queries;
- physical experiment;
- environment reward;
- independent model ensemble;
- human feedback for the final layer of difficult judgement.

When no external verifier exists, multiple independent evaluators should be used to reduce generator-verifier collusion.

5.3 Selection

The winner should not simply become the output. The result should update the **probability of future computation**.

For a sampled structure G_k , the system updates

$$\phi \leftarrow \phi + \alpha_{struct} \nabla_{\phi} \mathbb{E}[R_k],$$

while normal weights use their faster learning rate α_{weight} .

A successful rare path should become more likely, but not immediately deterministic.

6. Structural credit assignment

A major failure mode of naive plasticity is deleting useful but rarely activated components.

Each structural component therefore maintains a slowly changing credit estimate:

$$F_e(t) = \rho F_e(t-1) + (1-\rho) S_e(t),$$

where $S_e(t)$ estimates the component's marginal contribution.

A cheap first-order approximation is

$$S_e \approx \left| W_e \frac{\partial L}{\partial W_e} \right|,$$

but SPINE should use several signals where possible:

$$S_e = a \left| W_e \frac{\partial L}{\partial W_e} \right| + b \left| \frac{\partial R}{\partial \alpha_e} \right| + c \Delta R_{counterfactual} + d R_{future}.$$

The counterfactual term is especially important. For selected components, the system occasionally runs an ablation or intervention and observes the change in verified reward.

This creates a hierarchy of evidence:

gradient sensitivity < accumulated correlation < intervention < externally verified causal contribution.

The expensive measurements are used selectively rather than on every edge.

7. Birth, death, and rewiring

A topology event follows hysteresis rather than a one-step threshold.

For an edge:

$$F_e < \tau_{death} \text{ for sufficiently long} \Rightarrow e \text{ becomes dormant.}$$

Dormant capacity is not necessarily destroyed. It can remain available for regrowth.

Potential new edges are proposed based on learning signal:

$$P(e_{new} | x) \propto \exp\left(\frac{A_e}{T_{struct}}\right),$$

where A_e can combine gradient sensitivity, representational novelty, locality, and resource cost.

A small mutation probability allows exploration outside the currently high-probability neighborhood.

At the module level, a stable parent can spawn a child through a function-preserving or approximately function-preserving transformation, minimizing disruption. This is conceptually compatible with evolutionary architecture methods and growing-network methods [6,20,23].

8. The entropy controller

The proposal does **not** advocate constant high entropy.

Instead, SPINE maintains an adaptive entropy budget:

$$T(x) = T_{min} + (T_{max} - T_{min})\sigma(aU + bE + cN - d),$$

where:

- U = evaluator disagreement / epistemic uncertainty;
- E = recent prediction or task error;
- N = novelty or distribution shift.

When the system is confident and the task is familiar:

$$T \downarrow .$$

When it encounters an anomaly or poorly understood problem:

$$T \uparrow .$$

The intuition is a cognitive thermostat:

exploit when the model understands the problem; explore when the model detects that its current model is insufficient.

Recent work has shown that entropy can be used as a meaningful confidence signal during reasoning and that entropy-minimization can itself improve reasoning performance [24,25]. Those results motivate adaptive entropy control, but they do not establish the proposed exploration mechanism.

9. True randomness

A physical true-random source is proposed as an **optional substrate**, not as an assumption.

Let ξ be the entropy stream used to sample structural and reasoning states.

Two variants must be tested:

$$\xi_{pseudo} \text{ vs. } \xi_{physical}.$$

If pseudorandom bits produce identical learning dynamics, the important ingredient is stochastic computation, not physical randomness. That is the expected result under ordinary computational assumptions.

A positive result for physical randomness would therefore be extremely significant and would require careful controls against implementation artifacts, correlations, timing effects, and hidden state.

There is already hardware research on compact physical stochastic computing: photonic probabilistic bits and magnetic-tunnel-junction devices can provide physical stochasticity suitable for probabilistic computation [26,27].

The proposed scientific claim is deliberately narrow:

Physical randomness is worth investigating because it may provide a natural high-throughput entropy source, not because quantum or physical randomness is assumed to contain extra computational information.

10. Why not simply use ensembles?

An ensemble gives

$$f_1(x), f_2(x), \dots, f_K(x)$$

and combines their outputs.

SPINE instead gives

$$f(x; G_1), f(x; G_2), \dots, f(x; G_K)$$

where the **same learned system** contains a distribution over internal computations.

This has three potential advantages:

1. **Parameter efficiency:** alternative paths can share weights.
2. **Structural learning:** successful paths alter future probabilities.
3. **Open-ended capacity:** the system can allocate and reorganize structure instead of merely averaging fixed models.

A conventional ensemble is therefore an important baseline, not the target architecture.

11. Why not simply use MoE?

MoE learns routing over a mostly fixed set of experts.

SPINE adds three properties:

MoE $\rightarrow$ routing

while

SPINE $\rightarrow$ routing + structural plasticity + persistent selection.

An MoE expert that repeatedly becomes irrelevant can be downweighted, but SPINE can eventually remove its structural footprint and create new specialist capacity. More importantly, the routing distribution itself can be reshaped by verified outcomes across time.

12. Why this could matter for AGI

SPINE directly targets several limitations of static foundation models.

Continual learning

The model can retain useful changes in weights, topology, and memory rather than placing every new fact in external text memory.

Specialization

Rare capabilities can occupy small structural islands rather than requiring every computation to represent everything.

Open-ended discovery

The generator can create unusual candidates; the verifier can kill most of them while preserving rare successful ideas.

Adaptive compute

Easy problems can use a small stable subgraph; difficult problems can recruit more computation or explore more alternatives.

Architectural memory

Repeatedly useful structural pathways become more probable. Topology itself becomes a form of long-term memory.

Self-improvement

A system can improve the policy that selects and modifies computation, potentially creating an improvement loop similar in spirit to self-evolving agents, but inside the learned computational substrate.

None of these properties guarantees AGI. They are simply areas where the proposal addresses limitations that current research increasingly identifies as important.

13. A concrete training recipe

Phase A - Stable substrate

Train a conventional Transformer or MoE backbone to establish strong language representations. Do not attempt to evolve everything from a random graph immediately.

Phase B - Introduce latent structure

Attach learned stochastic gates to:

- attention heads;
- FFN blocks;
- expert routes;
- selected residual paths.

Use Concrete/Gumbel-Softmax initially.

Phase C - Structural plasticity

Convert selected gates into hard discrete masks. Start periodically rewiring a small percentage of capacity using gradient and accumulated credit. Introduce dormant capacity and controlled mutation.

Phase D - Generator/verifier training

Generate multiple internal trajectories. Score them with task losses and independent verification. Reward structural paths that reliably lead to verified outcomes.

Phase E - Adaptive entropy

Learn a context-dependent exploration temperature from uncertainty, error, novelty, and verifier disagreement.

Phase F - Test-time learning

Allow fast weights, memory, and selected structural probabilities to adapt during interaction. Require prospective validation before changes become persistent, following the general logic of recent prequential TTT work [22].

Phase G - Consolidation

Periodically freeze durable circuits, merge redundant modules, and compress unused capacity. This prevents permanent structural churn.

14. The central experimental program

The first experiment should deliberately be small.

Benchmark family

A. Algorithmic: parity, sorting, copying, recursion, graph algorithms, hidden-rule tasks.

B. ARC-style: tasks where the model must infer novel abstractions from few examples.

C. Code: generation with unit-test execution as a verifier.

D. Mathematics: search with symbolic or numerical verification.

E. Open-ended optimization: FunSearch-like heuristic discovery.

F. Continual learning: tasks arriving in shifting distributions over long sequences.

Baselines

1. Dense Transformer.
2. MoE Transformer.
3. Dense Transformer + sampling/self-consistency.
4. Transformer + verifier search.
5. Transformer + test-time training.
6. RigL-style dynamic sparse Transformer.
7. SPINE without structural birth/death.
8. Full SPINE.

Required ablations

Component	Ablation
stochastic internal state	deterministic internal state
learned topology	fixed sparse topology
generator	single trajectory
verifier	no verifier
entropy controller	fixed temperature
persistent adaptation	reset each episode
topology evolution	weights only
true randomness	cryptographic/pseudorandom RNG
verifier independence	shared generator/verifier weights

Primary metrics

Do not optimize only benchmark accuracy.

Measure:

verified discoveries / FLOP

adaptation gain / interaction

capability retained per active parameter

structural stability after consolidation

novel solutions per independent search

and continual-learning forgetting.

A particularly important metric is **novelty yield**: the number of distinct, independently verified solutions discovered per unit of inference or training compute.

15. Falsifiable predictions

The proposal should be considered unsuccessful if these effects cannot be demonstrated.

Prediction 1 - structural learning: under equal active FLOPs, learned topology should outperform a fixed topology on tasks requiring adaptation to new computational structure.

Prediction 2 - rare capability preservation: long-term fitness plus dormant regrowth should outperform instantaneous pruning on sparse-competence tasks.

Prediction 3 - controlled entropy: adaptive entropy should dominate a fixed temperature across a mixed benchmark containing familiar and out-of-distribution problems.

Prediction 4 - generator/verifier: a stochastic exploratory branch plus verifier should generate more unique verified solutions than either single-path reasoning or naive high-temperature sampling at equal compute.

Prediction 5 - structural memory: repeated exposure to a task family should cause the topology distribution to specialize, reducing future adaptation cost.

Prediction 6 - test-time plasticity: allowing a small subset of weights and topology probabilities to adapt at inference should improve sample efficiency on sequential tasks while preserving old capabilities through consolidation.

Prediction 7 - true randomness: physical randomness will either match pseudorandomness or demonstrate a measurable advantage after all implementation confounds are removed. Either outcome is scientifically useful.

16. Major risks and likely failure modes

16.1 The network never finds anything better

The search space may be too large, and gradient learning may remain superior at discovering useful solutions.

16.2 Structural churn destroys knowledge

Birth/death can become a source of catastrophic interference. Hysteresis, dormant capacity, and consolidation are therefore essential.

16.3 The generator exploits the verifier

This is not hypothetical. Prover-verifier research demonstrates that generators can learn to exploit weak evaluators [28]. Independent and adversarial verification is therefore central.

16.4 Hardware does not benefit

Unstructured dynamic sparsity can be slower despite lower mathematical FLOPs. Hardware-aware grouping is mandatory [8].

16.5 Entropy becomes noise

More randomness does not imply more creativity. Exploration must be coupled to useful selection pressure.

16.6 The system learns to game novelty

A naive novelty reward can generate meaningless diversity. Novelty should therefore receive positive reward only when accompanied by downstream value or verified progress.

16.7 True randomness adds nothing

This is entirely plausible. The proposal remains valuable if stochastic topology itself matters even when a PRNG is sufficient.

17. What would count as a breakthrough?

A breakthrough result would not be “SPINE gets 1.7% better accuracy.”

The strongest evidence would look like this:

At equal or lower compute, a model with learned stochastic topology discovers reusable computational structures that a fixed-topology model consistently fails to discover; after several problems, those structures reduce the cost of solving future unseen problems; and the resulting topology becomes interpretable as specialized reusable circuits.

An even stronger result would be **positive transfer of structure**:

$$Task_1 \rightarrow \text{new circuit}$$

then

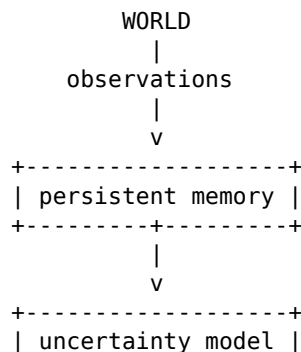
$$Task_2 \rightarrow \text{reuse} + \text{modify that circuit}$$

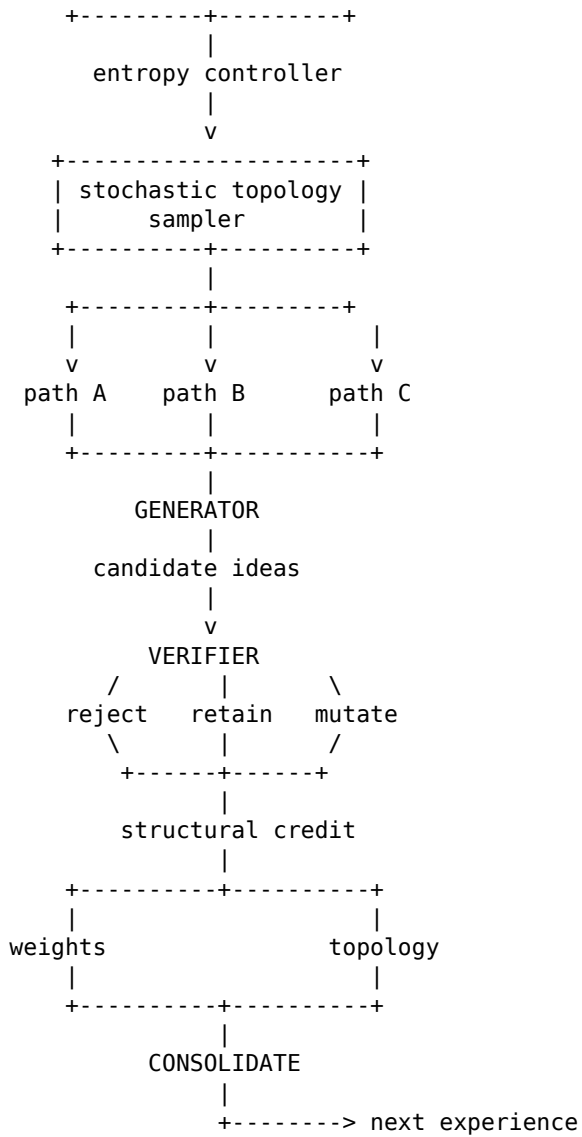
with lower adaptation cost than learning Task 2 from the original architecture.

That would show that topology is functioning as learned knowledge rather than merely as a compression mechanism.

18. A possible long-term architecture

A mature SPINE-like system could look conceptually like this:





The key property is that there is no clean boundary between “training” and “architecture design.” The architecture is continuously learned.

19. Why this is different from simply building a larger model

Scaling gives more capacity inside a known computational paradigm.

SPINE changes the optimization target itself:

learn θ

becomes

learn $(\theta, q(G), M, \pi_{adapt})$,

where:

- θ are ordinary parameters;
- $q(G)$ is the distribution over computations;
- M is persistent learned state;
- π_{adapt} is the policy deciding how the system changes itself.

That last term is particularly important. A system that learns **how to learn** is more powerful than one that merely learns a fixed update rule.

Recent work such as Meta-TTL makes this point explicitly by learning adaptation policies through outer-loop optimization rather than hand-designing them [29].

SPINE proposes to bring that principle inside the computational substrate.

20. Final proposition

The conventional mental model of a neural network is:

a machine whose parameters are learned.

The proposed mental model is:

a stochastic organism-like computation whose parameters, pathways, specialization, memory, and adaptation policy are all learned on different timescales.

The deepest hypothesis is therefore not “randomness is intelligence.” It is:

Intelligence may require learning a distribution over computations, not merely learning the parameters.
--

Under that view:

- hallucination becomes hypothesis generation;
- verification becomes selection;
- uncertainty becomes an entropy controller;
- weights become fast knowledge;
- topology becomes slower structural knowledge;
- birth/death becomes capacity allocation;
- rewiring becomes structural learning;
- test-time training becomes experience-driven adaptation;
- and randomness becomes the mechanism that explores the space of possible internal computations.

This is not known to be the route to AGI. It is a concrete, experimentally falsifiable alternative to the assumption that a mostly fixed neural substrate is sufficient.

The first step is not a trillion-parameter system.

The first step is a small model that can **change what it is made of in response to what it discovers**, while remaining measurably better than an equally powerful model that cannot.

If that happens, the research program becomes much more interesting.

References

[1] Maddison, Mnih, Teh. “The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables.” arXiv:1611.00712, 2016.

- [2] Jang, Gu, Poole. "Categorical Reparameterization with Gumbel-Softmax." arXiv:1611.01144, 2016.
- [3] Louizos, Welling, Kingma. "Learning Sparse Neural Networks through L0 Regularization." arXiv:1712.01312, 2017.
- [4] Molchanov, Ashukha, Vetrov. "Variational Dropout Sparsifies Deep Neural Networks." arXiv:1701.05369, 2017.
- [5] Bellec, Kappel, Maass, Legenstein. "Deep Rewiring: Training Very Sparse Deep Networks." arXiv:1711.05136, 2017.
- [6] Mocanu et al. "Scalable Training of Artificial Neural Networks with Adaptive Sparse Connectivity Inspired by Network Science." arXiv:1707.04780, 2017.
- [7] Evci et al. "Rigging the Lottery: Making All Tickets Winners." arXiv:1911.11134, 2020.
- [8] Lasby et al. "Dynamic Sparse Training with Structured Sparsity." arXiv:2305.02299, 2023.
- [9] Xie et al. "SNAS: Stochastic Neural Architecture Search." arXiv:1812.09926, 2018.
- [10] DeepSeek-AI et al. "DeepSeek-V3 Technical Report." arXiv:2412.19437, 2024.
- [11] Raposo et al. "Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models." arXiv:2404.02258, 2024.
- [12] Sun et al. "Learning to (Learn at Test Time)." arXiv:2310.13807, 2023.
- [13] Sun et al. "Learning to (Learn at Test Time): RNNs with Expressive Hidden States." arXiv:2407.04620, 2024.
- [14] Behrouz, Zhong, Mirrokni. "Titans: Learning to Memorize at Test Time." arXiv:2501.00663, 2025.
- [15] Tandon et al. "End-to-End Test-Time Training for Long Context." arXiv:2512.23675, 2025.
- [16] Yuksekgonul et al. "Learning to Discover at Test Time." arXiv:2601.16175, 2026.
- [17] Romera-Paredes et al. "Mathematical Discoveries from Program Search with a Large Language Model." Nature / FunSearch line of work, 2023-2024.
- [18] Mu, Zhang, Wang. "Planning of Heuristics: Strategic Planning on Large Language Models with Monte Carlo Tree Search for Automating Heuristic Optimization." arXiv:2502.11422, 2025.
- [19] Lin et al. "Leveraging Constrained Monte Carlo Tree Search to Generate Reliable Long Chain-of-Thought for Mathematical Reasoning." arXiv:2502.11169, 2025.
- [20] Zhang et al. "Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents." arXiv:2505.22954, 2025.
- [21] Chen et al. "Continual Learning in Large Language Models: Methods, Challenges, and Opportunities." arXiv:2603.12658, 2026.
- [22] Zhao et al. "Learn Now, Use Next, Trust Later: Prequential Test-Time Learning for LLM Agents." arXiv:2609.35911, 2026.
- [23] Du et al. "Efficient Network Construction through Structural Plasticity." arXiv:1905.11530, 2019.
- [24] Sharma, Chopra. "Think Just Enough: Sequence-Level Entropy as a Confidence Signal for LLM Reasoning." arXiv:2510.08146, 2025.
- [25] Agarwal et al. "The Unreasonable Effectiveness of Entropy Minimization in LLM Reasoning." arXiv:2505.15134, 2025.
- [26] Horodyski et al. "Stochastic Logic in Biased Coupled Photonic Probabilistic Bits." arXiv:2406.04000, 2024.

[27] Sun et al. “Superparamagnetic and Stochastic-Write Magnetic Tunnel Junctions for High-Speed True Random Number Generation in Advanced Computing.” arXiv:2509.13469, 2025.

[28] OpenAI-related prover-verifier work and subsequent generator/verifier research; see “Scalable AI Safety via Doubly-Efficient Debate,” arXiv:2311.14125 and related prover-verifier literature.

[29] Lou et al. “Learning to Learn-at-Test-Time: Language Agents with Learnable Adaptation Policies.” arXiv:2604.00830, 2026.

Research-status note

This document is a research proposal and synthesis, not a claim of established superiority. The literature review was performed as a targeted search of arXiv and adjacent primary sources available as of October 6, 2026. A complete prior-art search would require systematic citation-graph expansion and database-level search across ML, neuroscience, neuromorphic computing, probabilistic programming, and evolutionary computation.